

基于多级优先级队列的智能仓库任务调度仿真系统

一、实验项目名称

基于 LINKBOY 平台的多级优先级队列智能仓库任务调度仿真系统

二、实验目的

- 理解任务调度算法在物流仓储系统中的应用价值，掌握优先级调度、时间片轮转、最短作业优先等调度策略。
- 熟悉 LINKBOY 仿真平台中控制器、屏幕、矩阵键盘、无线遥控器、点阵屏、串口、灯带、舵机、四按钮、计时器等硬件的使用方法。
- 掌握多级队列、优先队列（堆）、任务控制块等数据结构在任务调度系统中的应用。
- 提高分析不同调度算法对系统吞吐量、平均等待时间、任务截止时间满足率等性能指标影响的能力。

三、系统需求分析

3.1 功能需求

1. 多优先级任务生成功能

- 系统模拟三种类型的仓库任务：**紧急订单**（优先级最高）、**普通补货**（优先级中等）、**低优先级巡检**（优先级最低）。
- 每种任务的到达间隔服从不同分布：~~紧急订单每 815 秒到达一个，普通补货每 38 秒到达一个~~，低优先级巡检每 10~20 秒到达一个。
- 每个任务包含：任务ID、任务类型、优先级（1/2/3，1最高）、预计执行时长（2~6秒）、到达时间戳。

2. 多级优先级队列调度功能

- 创建三个独立的队列，分别存放高、中、低优先级任务。
- 调度器按以下规则选择下一个执行的任务：
 - 严格优先级（非抢占式）：只要高优先级队列非空，总是执行高优先级任务；否则执行中优先级，最后低优先级。

- 加权轮询（可选模式）：可通遥控器切换为“加权轮询模式”，高/中/低队列分配 5:3:2 的时间片比例。
- 每次调度时，从选中的队列中取出队首任务执行。

3. 任务执行与资源占用模拟

- 使用**舵机**模拟任务执行机构：舵机转动到指定角度并保持一段时间表示“执行任务”，转动角度大小与任务类型关联（紧急订单 → 180°，普通补货 → 90°，低优先级巡检 → 0°）。
- 使用**灯带**颜色表示当前执行任务的优先级：红色=高优先级，黄色=中优先级，绿色=低优先级。
- 使用**屏幕**实时显示当前执行的任务ID、队列长度（三个队列）、调度模式、系统吞吐量等。

4. 调度策略切换与手动干预

- **无线遥控器（4键）**：
 - A键：切换调度模式（严格优先级 ↔ 加权轮询）。
 - B键：紧急暂停（当前任务执行完后停止新任务调度）。
 - C键：恢复调度。
 - D键：手动强制插入一个紧急订单（立即入高优先级队列）。
- **四按钮独立按键**：
 - 按钮1：记录并显示当前三个队列的任务快照（ID、等待时间）。
 - 按钮2：清空所有任务队列（紧急情况复位）。
 - 按钮3：切换屏幕显示模式（显示实时性能指标 vs 显示队列详情）。
 - 按钮4：发送当前系统性能统计报告到串口。

5. 矩阵键盘配置功能

- 通过矩阵键盘动态修改调度参数：
 - 数字键 + **A**：修改高优先级队列的最大长度（默认10）。
 - 数字键 + **B**：修改中优先级队列最大长度（默认15）。
 - 数字键 + **C**：修改低优先级队列最大长度（默认20）。
 - ***** 键：重置所有队列长度为默认值。
 - **#** 键：显示当前调度参数。

6. 数据统计与性能分析功能

- 系统自动记录以下指标（存储在列表中）：
 - 每个任务的**等待时间**（从入队到开始执行的时间差）。
 - 每个任务的**周转时间**（从入队到执行完成的时间差）。
 - 任务**完成数量**（按优先级分别统计）。

- 任务**丢弃数量**（队列满时无法入队的任务）。
- 每分钟自动计算一次平均等待时间、平均周转时间，并通过串口发送。

7. 串口通信与监控功能

- **上行数据**（系统 → 上位机）：
 - 实时发送每个任务开始执行和完成的信息（任务ID、类型、等待时间）。
 - 每分钟发送性能统计报告（平均等待时间、吞吐量）。
 - 队列溢出时发送报警信息。
- **下行命令**（上位机 → 系统）：
 - "STAT"：立即返回当前统计摘要。
 - "CLEAR"：清空所有统计记录。
 - "MODE <0/1>"：设置调度模式（0=严格优先级，1=加权轮询）。
 - "ADD <type>"：手动添加一个指定类型的任务（type=1/2/3）。

3.2 性能需求

- 任务调度响应时间 $\leq 200\text{ms}$ （从队列有任务到舵机开始动作）。
- 系统支持至少3个优先级队列，每个队列最大容量可配置（默认高10、中15、低20）。
- 任务到达间隔随机，模拟真实物流的不确定性。
- 系统连续仿真运行 ≥ 30 分钟无死锁或异常。
- 串口通信波特率 115200bps，数据格式 JSON 或 CSV。

3.3 用户角色与约束

- **调度员**：通过遥控器、按钮、键盘实时调整调度策略和参数。
- **管理员**：通过串口上位机远程监控系统性能，分析调度算法效果。
- **约束**：所有硬件基于 LINKBOY 平台仿真，采用图形化积木编程。

四、硬件设计

编号	模块名称	型号/规格	功能	GPIO分配
1	主控板	ESP32 / Arduino Mega	运行调度算法，控制所有外设	-
2	OLED显示屏	0.96" 128×64 (I ² C)	显示系统状态、队列长度、性能指标	SCL=22, SDA=21
3	灯带（RGB LED）	WS2812B 或 3个单色LED	指示当前执行任务的优先级	GPIO25
4	舵机	SG90	模拟任务执行机构（转动角度对应任务类型）	GPIO13
5	无线遥控器（4键）	虚拟模块	模式切换、暂停/恢复、紧急插入	中断引脚 GPIO15
6	四按钮独立按键	4× 轻触开关	快照、清空、显示切换、报告发送	GPIO26,27,28,29
7	矩阵键盘	4×4 矩阵键盘	参数配置	行: GPIO30-33, 列: GPIO34-37
8	串口（虚拟）	USB-Serial	与上位机通信	TX= 1, RX=0

硬件连接说明：

- 所有模块在 LINKBOY 仿真环境中拖拽放置，通过虚拟引脚连接。
- 无线遥控器使用“通用无线遥控器”模块，接收4个独立按键信号。
- 舵机需注意供电（仿真环境无需考虑实际电流）。

五、数据结构设计（图形化描述）

5.1 任务控制块（Task Control Block）

每个任务用一组变量表示，在 LINKBOY 中通过**列表（List）**存储多个任务。每个任务包含以下属性（存储在同一个列表的不同索引或分列表存储）：

字段名	数据类型	说明
taskID	整数	任务唯一编号，从1开始递增
type	整数（1~3）	1=紧急订单，2=普通补货，3=巡检
priority	整数（1~3）	1最高，3最低（与type相同）
execDuration	整数（毫秒）	预计执行时长（2000~6000ms）
arriveTime	整数	系统运行毫秒数（入队时刻）
startTime	整数	开始执行时刻（初始0）
finishTime	整数	完成时刻（初始0）

在 LINKBOY 中，可以创建 **7个列表**（每个属性一个列表），每个任务的相同索引位置对应同一任务；或者创建一个**任务列表**，每个元素是“任务ID”，其他属性通过单独的映射表存储。推荐前者，方便图形化操作。

5.2 多级队列结构

创建三个队列，每个队列用**列表**模拟，配合队首/队尾指针。由于 LINKBOY 列表操作方便，可以直接使用“列表末尾添加”表示入队，“删除列表第一项”表示出队，但为了模拟固定容量和性能，建议手动管理指针。

高优先级队列（Q_High）：

- 列表 `queueHigh` 存储任务ID（或完整任务副本）。
- 变量 `highHead` , `highTail` , `highCount` , `highMaxLen` 。

中优先级队列（Q_Mid）：

- 列表 `queueMid` ，指针变量类似。

低优先级队列（Q_Low）：

- 列表 `queueLow` ，指针变量类似。

队列基本操作伪代码：

子程序 入队(队列名, 任务ID):

如果 队列名.count < 队列名.maxLen:

将 任务ID 添加到 队列名.list 的末尾 // 简化操作, 直接使用列表追加

队列名.count += 1

返回 真

否则:

串口发送 "队列满, 丢弃任务 ID: " + 任务ID

丢弃计数 += 1

返回 假

子程序 出队(队列名):

如果 队列名.count > 0:

任务ID = 队列名.list[0] // 获取队首

删除 队列名.list[0]

队列名.count -= 1

返回 任务ID

否则:

返回 -1

5.3 统计信息存储

历史任务记录列表: 每个已完成任务存储一条字符串或分列表存储。记录格式:

- 任务ID、类型、等待时间 (ms)、周转时间 (ms)、是否被丢弃。

由于列表长度有限, 可设定最多保留最近 100 条记录。

性能指标变量:

- `totalTasksHigh`, `totalTasksMid`, `totalTasksLow`: 各优先级已完成任务数。
- `sumWaitTimeHigh`, `sumWaitTimeMid`, `sumWaitTimeLow`: 各优先级总等待时间 (用于计算平均值)。
- `totalDiscarded`: 总丢弃任务数。
- `lastReportTime`: 上次发送性能报告的时刻 (毫秒)。

5.4 随机数生成

- 任务到达间隔: 每种优先级独立计时器, 使用 `random()` 生成间隔毫秒数。
- 任务执行时长: `random(2000, 6000)` 毫秒。
- 随机种子在系统启动时固定 (如 `randomSeed(1234)`), 保证实验可重现。

5.5 调度模式与状态变量

- `schedMode`: 0 = 严格优先级 (非抢占), 1 = 加权轮询。

- `systemState` : 0 = 运行, 1 = 暂停 (暂停新任务调度, 当前任务可完成)。
- `rrCounter` : 加权轮询模式下的时间片计数器 (仅示意, 实际可用定时器)。
- `currentTaskID` : 当前正在执行的任务ID (若无则为-1)。
- `currentTaskRemainingTime` : 当前任务剩余执行时间 (毫秒)。

六、软件过程设计

6.1 主程序流程 (无限循环)

系统启动后执行一次:

1. 初始化所有硬件模块 (屏幕、串口、灯带、舵机归中)。
2. 初始化三个队列 (清空列表, 设置最大长度默认值)。
3. 初始化统计变量归零。
4. 设置随机种子 (固定值 1234)。
5. 设置系统状态为“运行”, 调度模式为“严格优先级”。
6. 设置三个优先级任务的“上次到达时间”和“下次间隔”变量。
7. 在屏幕上显示“系统就绪”。

然后重复执行以下步骤 (永不停止):

步骤1: 任务到达生成 (独立计时器, 每个优先级独立)

对于 `priority = 1,2,3`: 如果 系统状态 == 运行: 当前时间 = 运行毫秒数 如果 当前时间 - 上次到达时间[`priority`] $\geq$ 下次间隔[`priority`]: 任务ID 自增1 创建任务对象 (`ID`, `type=priority`, `priority`, `execDuration=随机(2000,6000)`, `arriveTime=当前时间`) 根据优先级调用对应的入队操作 如果入队成功: 在屏幕更新该队列长度显示 串口发送 "任务到达: ID=X, 类型=Y" 否则: 串口发送 "队列满, 丢弃任务 ID=X" 灯带橙色闪烁0.2秒 更新上次到达时间为当前时间 生成新的随机间隔 (毫秒): 高优先级: `random(8000,15000)` 中优先级: `random(3000,8000)` 低优先级: `random(10000,20000)`

步骤2: 调度与任务执行

如果 系统状态 == 运行 且 当前没有正在执行的任务 (`currentTaskID == -1`): 选择下一个任务 (根据 `schedMode`): 若 `schedMode == 0` (严格优先级): 如果 高优先级队列非空: 任务 = 出队(高) 否则如果 中优先级队列非空: 任务 = 出队(中) 否则如果 低优先级队列非空: 任务 = 出队(低) 否则: 无任务, 跳过 若 `schedMode == 1` (加权轮询): // 简化实现: 每轮依次检查三个队列, 但给高优先级更多机会 // 具体可采用时间片比例: 高队列每次最多取3个, 中队列2个, 低队列1个, 循环 实现一个简单的轮询指针, 按比例分配 如果有任务: `currentTaskID = 任务.ID` `currentTaskStartTime = 当前时间` 记录任务的开始时间到任务属性中 根据任务类型设置舵机目标

角度 (1→180°, 2→90°, 3→0°) 控制舵机转动 设置灯带颜色 (红/黄/绿) 设置一个定时器 (currentTaskRemainingTime = 任务的预计执行时长) 屏幕显示"执行任务 ID: X"

步骤3：任务执行完成检查

如果 currentTaskID != -1: 当前时间 = 运行毫秒数 如果 当前时间 - currentTaskStartTime ≥ currentTaskRemainingTime: 任务完成 记录完成时间, 计算等待时间 (startTime - arriveTime) 和 周转时间 (finishTime - arriveTime) 更新统计变量 (增加计数和总等待时间) 将任务记录存入历史列表 串口发送 "任务完成: ID=X, 等待时间=Y ms, 周转时间=Z ms" 舵机归中 (转到90°) 灯带恢复默认 (绿色常亮表示空闲) currentTaskID = -1

步骤4：遥控器信号处理

检查无线遥控器接收标志: A键: 切换 schedMode (0↔1), 屏幕显示当前模式, 串口发送模式切换信息。B键: systemState = 1 (暂停), 停止新任务调度 (当前任务继续完成)。C键: systemState = 0 (恢复), 并清除暂停标志。D键: 立即生成一个高优先级任务 (紧急订单), 强制入队高优先级队列, 忽略原到达间隔。

步骤5：四按钮独立按键处理

按钮1: 调用子程序"显示队列快照"(在屏幕上滚动显示各队列前5个任务的ID和等待时间)。按钮2: 清空所有任务队列 (清除列表, 重置队首队尾, 同时如果有正在执行的任务则让它完成但不从队列取新任务)。按钮3: 切换屏幕显示模式 (模式A: 实时性能指标; 模式B: 队列详情)。按钮4: 调用子程序"发送性能报告"到串口 (当前平均等待时间、已完成任务数、丢弃数)。

步骤6：矩阵键盘扫描

如果有键盘按键: 读取按键组合 (如数字+字母) 根据预设命令修改队列最大长度或调度参数 屏幕提示修改生效

步骤7：串口接收处理

如果串口有数据: 读取一行命令 (以换行结束) 解析命令: "STAT" → 立即发送统计摘要 (JSON格式) "CLEAR" → 清空所有历史统计记录, 重置计数器 "MODE 0" → 设置 schedMode = 0 "MODE 1" → 设置 schedMode = 1 "ADD 1" → 手动生成一个高优先级任务并入队 "ADD 2" → 手动生成一个中优先级任务 "ADD 3" → 手动生成一个低优先级任务 "GETQ" → 发送三个队列的当前长度

步骤8：周期性性能报告 (每分钟)

如果当前时间 - lastReportTime ≥ 60000 毫秒: 调用子程序"发送性能报告"(同按钮4)
lastReportTime = 当前时间

步骤9：屏幕刷新

每隔 200ms 刷新一次屏幕显示（避免闪烁）：显示当前调度模式、系统状态。显示三个队列长度。显示当前执行的任务ID及剩余时间。若在性能显示模式，显示平均等待时间和吞吐量。若在队列详情模式，显示各队列队首任务信息。

步骤10：循环延时

每次循环结束延时 10 毫秒，保持响应性。

6.2 各子模块详细伪代码

6.2.1 入队子程序（图形化实现参考）

子程序 入队(队列类型，任务对象)：

变量 队列列表 = 根据类型选择(queueHigh, queueMid, queueLow)

变量 最大长度 = 根据类型选择(highMaxLen, midMaxLen, lowMaxLen)

变量 当前长度 = 列表长度(队列列表)

如果 当前长度 < 最大长度：

将 任务对象 添加到 队列列表 末尾

返回 真

否则：

串口发送 "队列满，丢弃" + 任务对象.ID

全局丢弃计数 += 1

返回 假

6.2.2 出队子程序

子程序 出队(队列类型)：

变量 队列列表 = 根据类型选择

如果 列表长度(队列列表) > 0：

变量 任务 = 队列列表[0]

删除 队列列表[0] // 移除第一项

返回 任务

否则：

返回 空

6.2.3 严格优先级调度选择

```
子程序 选择任务_严格优先级():
    任务 = 空
    如果 列表长度(queueHigh) > 0:
        任务 = 出队(高)
    否则如果 列表长度(queueMid) > 0:
        任务 = 出队(中)
    否则如果 列表长度(queueLow) > 0:
        任务 = 出队(低)
    返回 任务
```

6.2.4 加权轮询调度选择（简化版）

```
变量 轮询计数器 = 0 // 全局变量，每调度一次递增

子程序 选择任务_加权轮询():
    任务 = 空
    轮询计数器 = (轮询计数器 + 1) % 6 // 6次一个周期

    // 比例：高:中:低 = 3:2:1
    如果 轮询计数器 < 3 且 列表长度(queueHigh) > 0:
        任务 = 出队(高)
    否则如果 轮询计数器 < 5 且 列表长度(queueMid) > 0:
        任务 = 出队(中)
    否则如果 列表长度(queueLow) > 0:
        任务 = 出队(低)
    否则:
        // 如果对应队列空，则尝试其他非空队列
        如果 列表长度(queueHigh) > 0: 任务 = 出队(高)
        否则如果 列表长度(queueMid) > 0: 任务 = 出队(中)
        否则如果 列表长度(queueLow) > 0: 任务 = 出队(低)
    返回 任务
```

6.2.5 统计报告生成

子程序 发送性能报告():

 计算平均等待时间 (高/中/低分别):

 如果 totalTasksHigh > 0: avgWaitHigh = sumWaitTimeHigh / totalTasksHigh

 否则 avgWaitHigh = 0

 同理中、低

 总体平均等待时间 = (sumWaitTimeHigh+sumWaitTimeMid+sumWaitTimeLow) /
(totalTasksHigh+totalTasksMid+totalTasksLow)

 吞吐量 = (totalTasksHigh+totalTasksMid+totalTasksLow) / (运行分钟数)

 组装字符串:

 "=== Performance Report ==="

 "High: tasks=XX, avgWait=XX ms"

 "Mid: tasks=XX, avgWait=XX ms"

 "Low: tasks=XX, avgWait=XX ms"

 "Overall avgWait=XX ms, Throughput=XX tasks/min"

 "Discarded=XX"

 通过串口发送

七、实验任务与考核要求

7.1 学生需完成的操作

1. 在 LINKBOY 平台图形化界面中按硬件设计连接所有模块，配置引脚。
2. 使用积木块编程实现上述伪代码描述的所有功能（至少完成严格优先级调度和加权轮询调度）。
3. 通过随机数生成三种优先级任务，观察队列长度的动态变化，验证调度算法的效果。
4. 使用串口助手监控系统输出的性能数据，并发送命令（STAT、MODE、ADD）观察系统响应。
5. 撰写实验报告，内容包括：
 - 系统架构图与数据结构设计说明。
 - 两种调度算法的流程图（积木截图）。
 - 对比两种调度算法在相同负载下的平均等待时间、吞吐量、高优先级任务响应时间。
 - 分析队列长度限制对任务丢弃率的影响。
 - 总结任务调度策略在物流系统中的应用价值。

7.2 创新扩展

- 实现**动态优先级调整**：任务等待时间超过阈值后自动提升优先级（防止饥饿）。
- 增加**任务截止时间**属性，实现最早截止时间优先（EDF）调度算法。
- 使用点阵屏模拟显示当前正在执行任务的“条形码”或ID。
- 增加 Wi-Fi 模块，将实时调度性能数据上传至云平台（图形化 MQTT 积木）。

- 实现多级反馈队列（MLFQ），任务在不同队列间迁移。
-